Production Node.js on AWS EC2

Architecture Justification (D2)

This document explains the key architectural decisions for the proposed deployment, the trade-offs considered, and the assumptions made. The accompanying diagram (D1) shows the full topology. The target region is ap-south-1 since the application is intended for Indian users; the design generalises to any region with two or more Availability Zones.

1. Networking and VPC

The design uses a custom VPC at 10.0.0.0/16 split across two AZs (1a and 1b) in a standard three-tier layout: public subnets for the ALB and NAT gateways, private subnets for the application instances, and isolated subnets reserved for the data tier (RDS, ElastiCache). The isolated subnets are provisioned as part of the VPC plan but the database and cache services themselves are scoped as future / optional components - this avoids re-engineering the VPC when the data layer is introduced.

Private subnets are allocated /22 to accommodate ASG growth; public subnets remain /24 since they only host ALB ENIs and NAT gateways. Subnet sizing was deliberately asymmetric to reflect long-term capacity needs.

Two NAT Gateways - one per AZ - have been chosen over a single shared NAT. A single NAT is approximately 50 percent cheaper but introduces a cross-AZ failure dependency: if the AZ hosting the shared NAT becomes unavailable, the other AZ loses egress and instances cannot reach AWS APIs or external dependencies. For a production-grade design, the additional cost of approximately USD 32 per month per gateway is justified by the resilience benefit. This is further offset by interface VPC endpoints for SSM and Secrets Manager, and a gateway endpoint for S3, which keep routine traffic off NAT entirely.

Security groups follow least-privilege and reference each other rather than CIDR blocks. The alb-sg permits 80 and 443 from the internet; app-sg permits the application port only from alb-sg; and the future db-sg will permit traffic only from app-sg. NACLs are kept close to default and function as a stateless secondary layer rather than a primary control. VPC Flow Logs are enabled at the VPC level and shipped to CloudWatch Logs with a 30-day retention policy, sufficient for incident triage without unnecessary storage cost.

2. Compute and Auto Scaling

EC2 instances run from a custom AMI built with Packer, containing Node.js 20 LTS, the CloudWatch agent, a systemd service definition, and log rotation. User-data is intentionally minimal - it fetches the latest application artifact and runtime configuration, then starts the service. The rationale is scale-out latency: an AMI that requires full dependency installation at boot typically takes three to four minutes to become healthy, whereas a baked AMI reduces this to under ninety seconds. This directly affects how responsively target tracking can react to traffic spikes.

The ASG is configured with min=2, desired=2, max=6. A minimum of two instances is enforced even under low load, since a single-instance deployment does not provide true high availability and the cost differential is not significant. Target tracking is set at 60 percent CPU to retain headroom for the time required for a new instance to warm up. Health checks are configured as ELB type with a grace period of 180 seconds; shorter grace periods have historically resulted in premature termination during Node.js cold starts.

Deployments use ASG Instance Refresh with a minimum healthy percentage of 50 and checkpoints, so a failed release affects only a portion of the fleet before the rollout is halted. A full blue/green deployment model was

considered but judged unnecessary for a stateless application of this scope; Instance Refresh provides comparable safety with significantly lower operational overhead.

3. Load Balancer, TLS, and WAF

An Application Load Balancer was chosen over a Network Load Balancer because Layer 7 features are required: path-based routing for future service expansion, HTTP health checks against /health, and native WAF integration. The HTTPS listener on port 443 terminates TLS using an ACM-issued certificate; port 80 is configured for redirection to HTTPS. ALB access logs are written to a dedicated S3 bucket with KMS encryption and a lifecycle policy (30 days Standard, 90 days Infrequent Access, 1 year Glacier, deletion at 7 years). Lifecycle policies on log buckets are an often-overlooked source of cost growth and are configured explicitly here.

AWS WAF is attached to the ALB as a WebACL rather than functioning as a separate network hop - this is reflected in the diagram. The rule set comprises the AWS Managed Core Rule Set, Known Bad Inputs, SQL injection rules, and a rate-based rule limiting requests to 2000 per five minutes per source IP. The rate threshold was selected based on prior experience where lower thresholds produced false positives against legitimate batch clients. WAF logs are delivered to S3 via Kinesis Firehose with sampling enabled.

Sticky sessions are disabled at the target group. The application is stateless by design; enabling stickiness would mask architectural concerns that should be addressed at the application layer rather than the load balancer.

4. Security, Secrets, and Access

SSH access is not permitted. EC2 instance access is provided exclusively through SSM Session Manager: port 22 is closed across all security groups, no key pair is attached via the launch template, and no bastion host is provisioned. Session activity is captured in CloudTrail, providing an auditable trail without additional tooling. The IAM instance profile includes the SSM core managed policy to enable this.

AWS Secrets Manager stores database credentials and API keys. The instance role grants read access scoped to specific secret ARNs rather than using wildcard permissions. Non-sensitive configuration that does not require rotation (feature flags, environment variables) is stored in SSM Parameter Store, which is more cost-effective for this category of data. Separate KMS Customer Managed Keys are used for EBS, S3, CloudWatch Logs, and Secrets Manager, with automatic key rotation enabled. Domain-specific keys preserve the ability to revoke or rotate one key without affecting unrelated resources.

IMDSv2 is enforced on the launch template. This mitigates a known class of SSRF-to-credential-exfiltration attacks and represents a low-effort, high-value security control.

5. Observability

The CloudWatch Unified Agent is installed on each EC2 instance and ships application logs (/app/nodejs), system logs (/system/syslog), and supplementary system metrics (memory, disk, swap) that are not provided by default CloudWatch metrics. The agent configuration is sourced from SSM Parameter Store, allowing updates without rebuilding the AMI.

Custom application metrics are emitted using CloudWatch Embedded Metric Format (EMF) directly from the Node.js application - request count per route, error count, and latency percentiles (p50, p95, p99). EMF was selected over a StatsD-based approach because it does not require an additional daemon and integrates natively with the existing log pipeline.

Alarms are wired to an SNS topic which fans out to email and a Slack channel via a Lambda subscriber. Primary alarms cover: sustained CPU above 80 percent, ALB 5xx errors exceeding 10 per minute, p95 latency above 1 second, application error rate above 1 percent, ASG unhealthy host count, and NAT Gateway port allocation errors. A single consolidated dashboard groups metrics by Traffic, Compute, Application, Errors, and WAF activity, optimised for use during incident response.

CloudTrail is configured as a multi-region trail with log file integrity validation enabled, writing to an S3 bucket with Object Lock for tamper protection. This represents the baseline audit configuration applied across all environments.

6. Infrastructure as Code and Environments

All infrastructure is provisioned through Terraform using a modular structure with discrete modules for VPC, ALB, ASG, WAF, security, and observability. Terraform state is stored in S3 with DynamoDB-based state locking. Static analysis (tflint, tfsec) is executed in CI prior to any plan or apply operation. Staging and production environments share the same modules but consume separate tfvars files; the significant differences are instance sizing, log retention windows, and NAT Gateway count (one in staging for cost efficiency, two in production for high availability).

Resource tagging is enforced through a common locals block merged at the module level, applying Environment, Project, Owner, CostCenter, and ManagedBy tags consistently. This supports downstream cost allocation and operational discoverability.

A CI/CD pipeline is listed as optional in the brief. The intended approach is a Jenkins pipeline executing terraform plan and policy checks on pull requests for infrastructure changes, with application deployments handled through ASG Instance Refresh - the launch template is updated with a new AMI or user-data version, and Instance Refresh performs the rolling replacement. An alternative would be AWS CodeDeploy in in-place or blue/green mode; the choice between the two depends on release strategy requirements, with Instance Refresh preferred here for its operational simplicity on stateless workloads.

7. Trade-offs Accepted

- Two NAT Gateways instead of one: approximately USD 32 per month additional cost, but removes a cross-AZ failure mode.
- Custom AMI baking adds a Packer build stage to the pipeline, slowing iteration on base image changes but reducing scale-out latency by more than half.
- No CloudFront in front of the ALB: the assignment scope is regional API traffic rather than global static content. CloudFront can be added later with minimal architectural impact.
- Isolated subnets for the data tier are provisioned but unused at this stage. This incurs a small overhead now but avoids re-planning the VPC when RDS and ElastiCache are introduced.
- Burstable t3.medium instances are used rather than fixed-performance m-family instances: lower baseline cost, sufficient for typical Node.js API workloads. CPU credit metrics will be monitored and the choice revisited if credit exhaustion is observed.

8. Assumptions

- The application is stateless: no in-memory session data, no reliance on local filesystem state. Any stateful concerns are deferred to the future data tier.

- Traffic profile is variable but not extreme. CPU-based target tracking is sufficient; the design is not optimised for sudden 100x traffic spikes.
- A single AWS account is used initially, with environment separation managed through Terraform workspaces and tfvars files. A multi-account structure (for example, AWS Control Tower) is the longer-term direction but is out of scope.
- The domain is already registered and managed within Route 53.
- RDS, ElastiCache Redis, and any other data-tier services are treated as optional / future components. The VPC plan accommodates them but they are not provisioned in the current scope.
- The target region is ap-south-1 (Mumbai). The design is portable to any region with two or more Availability Zones.

9. Future Enhancements

Planned next steps, in approximate priority order: (1) provisioning RDS Multi-AZ within the isolated subnets when the data tier is required; (2) establishing a full CI/CD pipeline for application deployments; (3) cross-region disaster recovery planning, including documented RTO and RPO targets and a cold standby strategy; (4) cost anomaly detection and a structured FinOps review, with particular attention to NAT Gateway and CloudWatch Logs spend; (5) controlled chaos engineering exercises once the platform has reached steady-state operation, to validate failure-mode assumptions in production conditions.

— End of document —